Discussion 2

MSBA 402

Welcome

What discussion can covers:

- Lecture recap
- Practice problems
- Quiz review
- Technical support

Also utilize Piazza and office hours.

Email fauzan.prasetya@ucla.edu

Zoom link: https://ucla.zoom.us/my/fauzan.prasetya?pwd=JQLRVksvXRZVxk46xBTUTYaIa2Nwau.1

TA office hours on Thursday 11am to 12pm

Today’s discussion overview

This discussion will cover these topics:

- Data types lecture recap
- Quiz 1 review
- Intro to Data Manipulation Language (DML) query (*optional*)

Data types overview (numeric)

Category	Type Name	Description	Example Values
Numeric	serial (serial4)	Standard 4-byte auto Incrementing integer (often used for primary keys) Up to 2,147,483,647	1, 2, 3
	bigserial (serial8)	8-byte auto Incrementing integer	3,147,483,647
	integer (int, int4)	Standard 4-byte whole number (- 2,147,483,648 to 2,147,483,647)	42, -105, 2024
	smallint (int2)	2-byte whole number for small values (-32,768 to 32,767)	12, 32764
	bigint (int8)	8-byte whole number for large values	9223372036854775807

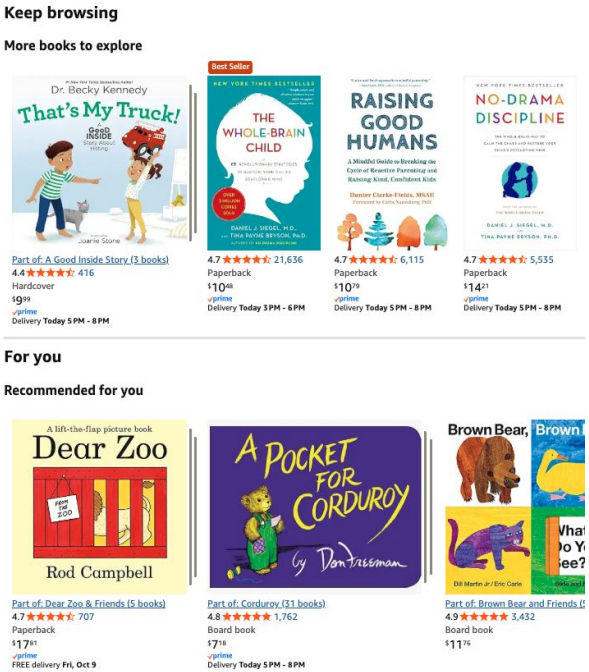
Byte is size for storage. Bigger byte → Need more space

Adapted from PostgreSQL documentation
https://www.postgresql.org/docs/current/datatype.html

Exercise 1

We are building a database for online bookstore. What data type would you use for these columns?

- Book ID
- Page Count
- Total Global Downloads
- Publication Year



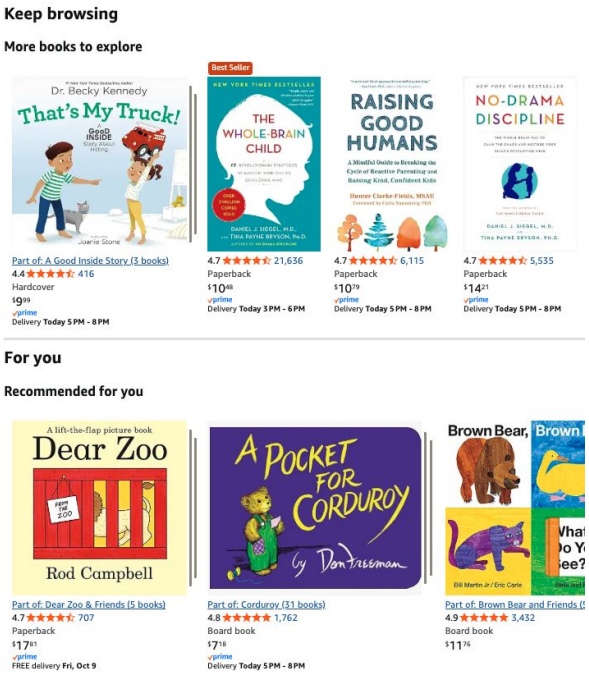
For reference

- serial (serial4)
- bigserial (serial8)
- integer (int, int4)
- smallint (int2)
- bigint (int8)

Exercise 1

We are building a database for online bookstore. What data type would you use for these columns?

- Book ID → serial
- Page Count → smallint
- Total Global Downloads → bigint
- Publication Year → smallint



For reference

- serial (serial4)
- bigserial (serial8)
- integer (int, int4)
- smallint (int2)
- bigint (int8)

Data types overview (numeric decimals)

Category	Type Name	Description	Example Values
Numeric	real (float4)	4-byte floating-point number	1.1234568
	double precision (float8)	8-byte floating-point number	1.1234567890123457
	decimal [p, s]	Exact numeric with user-specified precision (common for currency) *[precision, scale] → [total number of digits, digits after decimal points]*	19.99, 100.00 decimal [4,2], decimal [5,2]

Floats (real and double precision) are prone to rounding errors.

They are not recommended for financial data.

Example: 0.1 + 0.2 might yield 0.30000000000000004 instead of an exact 0.3

Adapted from PostgreSQL documentation
https://www.postgresql.org/docs/current/datatype.html

Exercise 2

We are building a database for online bookstore. What data type would you use for these columns?

- Book ID → serial
- …
- Customer Rating, e.g. 4.85
- Book Price, e.g. $19.99
- Store GPS Location (long, lat)

For reference

- real (float4)
- double precision (float8)
- decimal [p, s]

Exercise 2

We are building a database for online bookstore. What data type would you use for these columns?

- Book ID → serial
- …
- Customer Rating, e.g. 4.85 → real
- Book Price, e.g. $19.99 → decimal [4,2]
- Store GPS Location (long, lat) → double precision

For reference

- real (float4)
- double precision (float8)
- decimal [p, s]

Data types overview (character, boolean)

Category	Type Name	Description	Example Values
Character	text	Variable unlimited length string	'Hello, world!', 'Detailed description text'
	varchar(n)	Variable-length string with a maximum limit n	'Drake'
	char(n)	Fixed-length string, blank-padded	'US', 'CA', 'NY' *(for char(2))*
Boolean	boolean (or bool)	Logical Boolean value	TRUE, FALSE

VARCHAR only stores the actual characters provided → More efficient by size

CHAR always pads the value with trailing spaces to fill the defined length n.

- VARCHAR(5) or CHAR(5): Uses 5 characters (stores 'Drake').
- VARCHAR(10): Uses 5 characters (stores **'Drake',** without adding extra spaces).
- CHAR(10): Uses 10 characters (stores **'Drake** **'** with 5 trailing spaces).

Use varchar instead of text, for data integrity based on characters limits

Example: VARCHAR(8) for license plate numbers

Adapted from PostgreSQL documentation
https://www.postgresql.org/docs/current/datatype.html

Exercise 3

We are building a database for online bookstore. What data type would you use for these columns?

- Book ID → serial
- …
- In Stock Status
- ISO Country Code: e.g. 'US', 'JP'
- Book Title
- Full Book Description

For reference

- text
- varchar(n)
- char(n)
- boolean (or bool)

Exercise 3

We are building a database for online bookstore. What data type would you use for these columns?

- Book ID → serial
- …
- In Stock Status → boolean
- ISO Country Code: e.g. 'US', 'JP' → char(2) or varchar(2)
- Book Title → varchar(150)
- Full Book Description → text

For reference

- text
- varchar(n)
- char(n)
- boolean (or bool)

Data types overview (timestamp)

Category	Type Name	Description	Example Values
Date / Time	date	Calendar date (year, month, day)	'2026-10-06'
	time	Time of day	'14:30:00'
	timestamp	Date and time (without time zone)	'2026-10-06 14:30:00'
	timestamptz	Date and time, with time zone awareness	'2026-10-06 14:30:00-07', '2026-10-06 14:30:00+05:30' '2026-10-06 14:30:00 PST' '2026-10-06 14:30:00 Asia/Tokyo'
	interval	Time span / duration	'3 days 4 hours', '1 month', '3 weeks'

Always use **timestamptz** instead of timestamp.

To get current timestamp → use CURRENT_TIMESTAMP function

Detail timestamp documentation:
https://www.postgresql.org/docs/current/datatype-datetime.html

Adapted from PostgreSQL documentation
https://www.postgresql.org/docs/current/datatype.html

Data types overview (identifier and json)

Category	Type Name	Description	Example Values
Identifier	uuid	Universally Unique Identifier	'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11'
JSON	json	Exact copy of the plain text JSON data	'{"user_id": 101, "roles": ["admin", "editor"]}'
	jsonb	Binary JSON data	'{"user_id": 101, "roles": ["admin", "editor"]}'

JSON is used when we need a **large data**, or we want to **add more data without changing the table schema** → no need to add new column

jsonb is more efficient for querying

Adapted from PostgreSQL documentation
https://www.postgresql.org/docs/current/datatype.html

json examples

```
{
  "order_id": "ORD-2026-98765",
  "status": "shipped",
  "customer": {
    "name": "Sarah Connor",
    "email": "sarah.c@example.com",
    "is_prime": true
  },
  "shipping_address": {
    "street": "742 Evergreen Terr",
    "city": "Springfield",
    "state": "OR",
    "zip": "97477"
  },
  "payment": {
    "method": "credit_card",
    "card_last_4": "4242",
    "total_paid": 129.60,
    "is_paid": true
  },
```

```
  "items": [
    {
      "sku": "HEADPHONES-BLK",
      "name": "Wireless Headphones",
      "qty": 1,
      "price": 90.00
    },
    {
      "sku": "CABLE-USB-C",
      "name": "USB-C Fast Cable",
      "qty": 2,
      "price": 15.00
    }
  ],
  "gift_wrap": {
    "is_gift": true,
    "message": "Happy Birthday!"
  },
  "tracking_updates": [
    {"time": "10:15", "status": "Order Placed"},
    {"time": "18:00", "status": "Out for Delivery"}
  ]
}
```

Data types overview (identifier and json)

Category	Type Name	Description	Example Values
Identifier	uuid	Universally Unique Identifier	'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11'
JSON	json	Exact copy of the plain text JSON data	'{"user_id": 101, "roles": ["admin", "editor"]}'
	jsonb	Binary JSON data	'{"user_id": 101, "roles": ["admin", "editor"]}'

JSON is used when we need a **large data**, or we want to **add more data without changing the table schema** → no need to add new column

jsonb is more efficient for querying

Adapted from PostgreSQL documentation
https://www.postgresql.org/docs/current/datatype.html

Category: ENUM

Value can be only one value from a fixed list.

Also define orders when it is created, so small < large

```
-- Define a custom ENUM data type
CREATE TYPE tshirt_size AS ENUM (
    'small',
    'medium',
    'large',
    'xlarge'
);

-- Create table using the custom ENUM type
CREATE TABLE person (
    student_name VARCHAR(255),
    tshirt_size  tshirt_size
);
```

Exercise 4

We are building a database for online bookstore. What data type would you use for these columns?

- Book ID → serial
- …
- Daily Store Opening Time
- Date Book Published
- Order Checkout Timestamp
- Typical Shipping Duration e.g., '3 days' or '6 hours'
- Book Format e.g., 'hardcover', 'paperback', 'audiobook', 'ebook'

For reference

- date
- time
- timestamp
- timestamptz
- interval
- enum

Exercise 4

We are building a database for online bookstore. What data type would you use for these columns?

- Book ID → serial
- …
- Daily Store Opening Time → time
- Date Book Published → date
- Order Checkout Timestamp → timestamptz
- Typical Shipping Duration e.g., '3 days' or '6 hours' → interval
- Book Format e.g., 'hardcover', 'paperback', 'audiobook', 'ebook' → enum

For reference

- date
- time
- timestamp
- timestamptz
- interval
- enum

Quiz 1 review

Question 1

The school uses these rules for section:

- A course is offered in many terms. Within a term, the sections of a course are numbered 1, 2, 3, ...
- In a given term, a room holds at most one section at each start time. (Ignore days of the week.)
- An instructor can teach many sections. Two sections of the same course can meet at the same time in different rooms.

For items (a) through (e), use the two tables and the rules above.

course

course_code	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

section

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	27W	1	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

Quiz 1 review

Question 1

The school uses these rules for section:

- **A course is offered in many terms. Within a term, the sections of a course are numbered 1, 2, 3, ...**
- In a given term, a room holds at most one section at each start time. (Ignore days of the week.)
- An instructor can teach many sections. Two sections of the same course can meet at the same time in different rooms.

For items (a) through (e), use the two tables and the rules above.

`course as Parent (Referred)`

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

PK `section as Child (Referring)`

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	27W	1	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

Quiz 1 review

Question 1

course as Parent (Referred)

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

PK section as Child (Referring)

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	27W	1	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

(a) No two of the four rows of section have the same instructor_email. The only statement about {instructor_email} as a key of section that is true is —

- [ ] It is a superkey, because no two of these four rows share an email.
- [ ] It is a superkey, because each instructor has a different email address.
- [ ] It is not a superkey, because an instructor can teach many sections.
- [ ] It is not a superkey, because a key must have more than one column.

The school uses these rules for section:

- A course is offered in many terms. Within a term, the sections of a course are numbered 1, 2, 3, ...
- In a given term, a room holds at most one section at each start time. (Ignore days of the week.)
- **An instructor can teach many sections.** Two sections of the same course can meet at the same time in different rooms.

For items (a) through (e), use the two tables and the rules above.

Quiz 1 review

Question 1

course as Parent (Referred) PK section as Child (Referring)

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	27W	1	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	**1**	CORNELL-D313	**09:00**	**dee@ucla.edu**
MGMT403	26F	**2**	CORNELL-D313	**13:00**	**dee@ucla.edu**

(a) No two of the four rows of section have the same instructor_email. The only statement about {instructor_email} as a key of section that is true is —

- [ ] It is a superkey, because no two of these four rows share an email.
- [ ] It is a superkey, because each instructor has a different email address.
- **[ ] It is not a superkey, because an instructor can teach many sections.**
- [ ] It is not a superkey, because a key must have more than one column.

A superkey is **any set** (**can be single or multiple columns**) of columns that **uniquely** identifies each row.

The school uses these rules for section:

- A course is offered in many terms. Within a term, the sections of a course are numbered 1, 2, 3, ...
- In a given term, a room holds at most one section at each start time. (Ignore days of the week.)
- **An instructor can teach many sections.** Two sections of the same course can meet at the same time in different rooms.

For items (a) through (e), use the two tables and the rules above.

Quiz 1 review

Question 1

course as Parent (Referred)

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

PK section as Child (Referring)

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	27W	1	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

(b) The primary key of section is (course_code, term, section_num). Using the rules, **write the clause** in CREATE TABLE that declares the **other candidate key** of section.

A candidate key is **a minimal** superkey.

The school uses these rules for section:

- A course is offered in many terms. Within a term, the sections of a course are numbered 1, 2, 3, ...
- In a given term, a room holds at most one section at each start time. (Ignore days of the week.)
- An instructor can teach many sections. Two sections of the same course can meet at the same time in different rooms.

For items (a) through (e), use the two tables and the rules above.

Quiz 1 review

Question 1

course as Parent (Referred)

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

PK section as Child (Referring)

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	27W	1	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

(b) The primary key of section is (course_code, term, section_num). Using the rules, **write the clause** in CREATE TABLE that declares the **other candidate key** of section.

A candidate key is **a minimal** superkey.

{term, room, start time}

term	room	start_time
26F	GOLD-B201	09:00
26F	GOLD-B201	13:00
27W	GOLD-B201	09:00
26F	CORNELL-D313	09:00

The school uses these rules for section:

- A course is offered in many terms. Within a term, the sections of a course are numbered 1, 2, 3, ...
- **In a given term, a room holds at most one section at each start time.** (Ignore days of the week.)

A room cannot be used for two sections at the same time in the same term

For items (a) through (e), use the two tables and the rules above.

Quiz 1 review

Question 1

course as Parent (Referred)

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

PK section as Child (Referring)

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	27W	1	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

(b) The primary key of section is (course_code, term, section_num). Using the rules, **write the clause** in CREATE TABLE that declares the **other candidate key** of section.

A candidate key is **a minimal** superkey.

UNIQUE (term, room, start time)

```
CREATE TABLE section (
    course_code      VARCHAR(10) NOT NULL,
    term             VARCHAR(10) NOT NULL,
    section_num      INTEGER NOT NULL,
    room             VARCHAR(20) NOT NULL,
    start_time       TIME NOT NULL,
    instructor_email VARCHAR(255) NOT NULL,

    -- Primary Key declaration
    PRIMARY KEY (course_code, term, section_num),

    -- Foreign Key referencing the course table
    FOREIGN KEY (course_code) REFERENCES course (course_code),

    -- Candidate Key declaration as requested in 1(b)
    UNIQUE (term, room, start_time)
);
```

Quiz 1 review

Question 1

`course as Parent (Referred)`

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

`section as Child (Referring)`

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	27W	1	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

(a) No two of the four rows of section have the same instructor_email. The only statement about {instructor_email} as a key of section that is true is —

- **[ ] It is a superkey, because no two of these four rows share an email.**
- ~~[ ] It is a superkey, because each instructor has a different email address.~~
- [ ] It is not a superkey, because an instructor can teach many sections.
- [ ] It is not a superkey, because a key must have more than one column.

By only using four rows, we should think that section table is a **static and isolated table**.

Those four rows are **the only rows they have**.

By applying that logic, can we use **instructor_email** as superkey?

Quiz 1 review

Question 1

course as Parent (Referred)

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

PK section as Child (Referring)

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	27W	1	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

(b) The primary key of section is (course_code, term, section_num). Using the rules, **write the clause** in CREATE TABLE that declares the **other candidate key** of section.

A candidate key is **a minimal** superkey.

UNIQUE (instructor_email) → this works

Quiz 1 review

Question 1

course as Parent (Referred)

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

PK section as Child (Referring)

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	27W	1	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

(c) A colleague proposes two foreign keys:

(i) section.course_code references course.course_code

(ii) course.course_code references section.course_code

Which one is correct? Give one reason why the other one cannot work.

Quiz 1 review

Question 1

`course as Parent (Referred)`

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

`section as Child (Referring)`

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	27W	1	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

(c) A colleague proposes two foreign keys:

(i) section.course_code references course.course_code

~~(ii) course.course_code references section.course_code~~

child_table REFERENCES parents_table

Which one is correct? Give one reason why the other one cannot work.

Course should exist before section. Course as parents or referred table, and section as child or referring table. Column to be referred must unique.

Option (ii) fails because **section.course_code is not unique**, so course.course_code cannot reference it.

Quiz 1 review

Question 1

(d) A table enrollment records which students are in which sections. It has FOREIGN KEY (course_code, term, section_num) REFERENCES section (course_code, term, section_num). Using the four rows of section, the only value of (course_code, term, section_num) that can be stored in enrollment is

- [ ] (MGMT402, 27W, 1)
- [ ] (MGMT402, 26F, 3)
- [ ] (MGMT403, 27W, 1)
- [ ] (MGMT404, 26F, 1)

Records on parent should be exist before child.

PK `section as Parent (Referred)`

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	**27W**	**1**	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

`enrollment as Child (Referring)`

enrollment_id	student_id	course_code	term	section_num
1	11	MGMT402	26F	1
2	22	MGMT402	26F	2
3	33	MGMT402	27W	1
4	44	MGMT403	26F	1

Quiz 1 review

Question 1

(d) A table enrollment records which students are in which sections. It has FOREIGN KEY (course_code, term, section_num) REFERENCES section (course_code, term, section_num). Using the four rows of section, the only value of (course_code, term, section_num) that can be stored in enrollment is

- **[ ] (MGMT402, 27W, 1)**
- ~~[ ] (MGMT402, 26F, 3)~~
- ~~[ ] (MGMT403, 27W, 1)~~
- ~~[ ] (MGMT404, 26F, 1)~~

Records on parent should be exist before child.

PK `section as Parent (Referred)`

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	**27W**	**1**	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

`enrollment as Child (Referring)`

enrollment_id	student_id	course_code	term	section_num
1	11	MGMT402	26F	1
2	22	MGMT402	26F	2
3	33	MGMT402	27W	1
4	44	MGMT403	26F	1

Quiz 1 review

course as Parent (Referred)

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

PK section as Child (Referring)

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	27W	1	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

Question 1

(e) Assume only the correct foreign key from (c) exists, with no ON DELETE or ON UPDATE clause. Each change starts from the original data above. Mark whether each change is allowed or rejected.

Records on parent should be exist before child.

i. Add a **section** of course MGMT999.

- [ ] Rejected
- [ ] Allowed

ii. Delete course MGMT404 from **course**.

- [ ] Rejected
- [ ] Allowed

Quiz 1 review

course as Parent (Referred)

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

PK section as Child (Referring)

course_code	term	section_num	room	start_time	instructor_email
MGMT402	26F	1	GOLD-B201	09:00	ana@ucla.edu
MGMT402	26F	2	GOLD-B201	13:00	ben@ucla.edu
MGMT402	27W	1	GOLD-B201	09:00	chen@ucla.edu
MGMT403	26F	1	CORNELL-D313	09:00	dee@ucla.edu

Question 1

(e) Assume only the correct foreign key from (c) exists, with no ON DELETE or ON UPDATE clause. Each change starts from the original data above. Mark whether each change is allowed or rejected.

i. Add a **section** of course MGMT999.

- **[ ] Rejected** → MGMT999 **does not exist** in the **parent** course table
- [ ] Allowed

ii. Delete course MGMT404 from **course**.

- [ ] Rejected
- [ ] **Allowed** → No child rows depend on MGMT404

Records on parent should be exist before child.

Quiz 1 review

Question 2

2. Keys and Data Types.
For items (a) through (c), choose the best answer.

(a) A table member is created with member_id INTEGER PRIMARY KEY and email VARCHAR(100) UNIQUE. The only statement that is true is —

- [ ] Two rows can both have a NULL member_id, as long as their emails are different.
- [ ] Two rows can both have a NULL email.
- [ ] Two rows can have the same email, as long as their member_id values are different.
- [ ] Because member_id is the primary key, PostgreSQL fills it in automatically (1, 2, 3, ...) when a row is added without one.

`table`

member_id (PK)	**Email** (UNIQUE)
1	aa@gmail.com
2	bb@gmail.com
3	cc@gmail.com
4	dd@gmail.com

Quiz 1 review

Question 2

2. Keys and Data Types.
For items (a) through (c), choose the best answer.

(a) A table member is created with member_id INTEGER PRIMARY KEY and email VARCHAR(100) UNIQUE. The only statement that is true is —

- [ ] Two rows can both have a NULL member_id, as long as their emails are different. → PK is never null
- **[ ] Two rows can both have a NULL email.** → UNIQUE can be null
- [ ] Two rows can have the same email, as long as their member_id values are different. → UNIQUE should be unique
- [ ] Because member_id is the primary key, PostgreSQL fills it in automatically (1, 2, 3, ...) when a row is added without one. → data type need to be serial or need another clause (generated always as identity)

`table`

member_id (PK)	**Email** (UNIQUE)
1	aa@gmail.com
2	bb@gmail.com
3	cc@gmail.com
4	dd@gmail.com

UNIQUE a rule that makes sure all values in a column, are completely different across every row in a table

Quiz 1 review

Question 2

2. Keys and Data Types.

For items (a) through (c), choose the best answer.

(b) A column stores exact dollar amounts such as 1049.99. The largest possible value is **99999.99**. The best data type for this column is —

- [ ] DECIMAL(7,2)
- [ ] DECIMAL(5,2)
- [ ] REAL
- [ ] DOUBLE PRECISION

[precision, scale]
→ [total number of digits, digits after decimal points]

Quiz 1 review

Question 2

2. Keys and Data Types.

For items (a) through (c), choose the best answer.

(b) A column stores exact dollar amounts such as 1049.99. The largest possible value is **99999.99**. The best data type for this column is —

- **[ ] DECIMAL(7,2)**
- [ ] DECIMAL(5,2)
- [ ] REAL
- [ ] DOUBLE PRECISION

[precision, scale]
→ [total number of digits, digits after decimal points]

[7 total number of digits, 2 digits after decimal points]

Quiz 1 review

Question 2

2. Keys and Data Types.
For items (a) through (c), choose the best answer.

(c) (1 point) A social network has users in Los Angeles and in London. Each post records the exact moment it was created, and **each user should see that moment in their own time zone**. The column should use the type —

- [ ] DATE
- [ ] TIMESTAMP
- [ ] INTERVAL
- **[ ] TIMESTAMPTZ**

CREATE TABLE

```
-- Create table
CREATE TABLE courses (
    course_code VARCHAR(10) PRIMARY KEY,
    title VARCHAR(100) NOT NULL
);
-- Insert initial records
INSERT INTO courses (course_code, title)
VALUES
    ('MGMT402', 'SQL'),
    ('MGMT403', 'Statistics'),
    ('MGMT404', 'Marketing');
```

courses table

course_code (PK)	title



courses table

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

INSERT

```
INSERT INTO courses (course_code, title)

VALUES ('MGMT405', 'Data Management');
```

courses table

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing
MGMT405	Data Management

UPDATE

```
UPDATE courses

SET title = 'Data'

WHERE course_code = 'MGMT405';
```

courses table

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing
MGMT405	Data

DELETE

```
DELETE FROM courses

WHERE course_code = 'MGMT405';
```

courses table

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

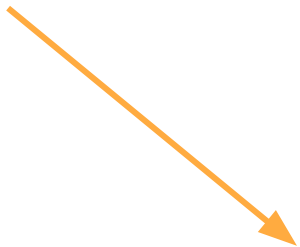
SELECT (1)

```
SELECT course_code

FROM courses

ORDER BY course_code DESC;
```

The original table won't change.



courses table

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

Query result

course_code
MGMT404
MGMT403
MGMT402

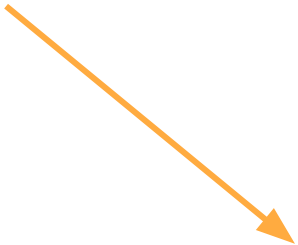
SELECT (2)

```
SELECT course_code, title

FROM courses

WHERE title = 'SQL';
```

The original table won’t change.



courses table

course_code (PK)	title
MGMT402	SQL
MGMT403	Statistics
MGMT404	Marketing

Query result

course_code	title
MGMT402	SQL